



Преподаватель:

Коляда

Никита Владимирович

Обратная связь:

- сообщения на inStudy

HTTP ПРОТОКОЛ И ХРАНЕНИЕ СОСТОЯНИЯ

—

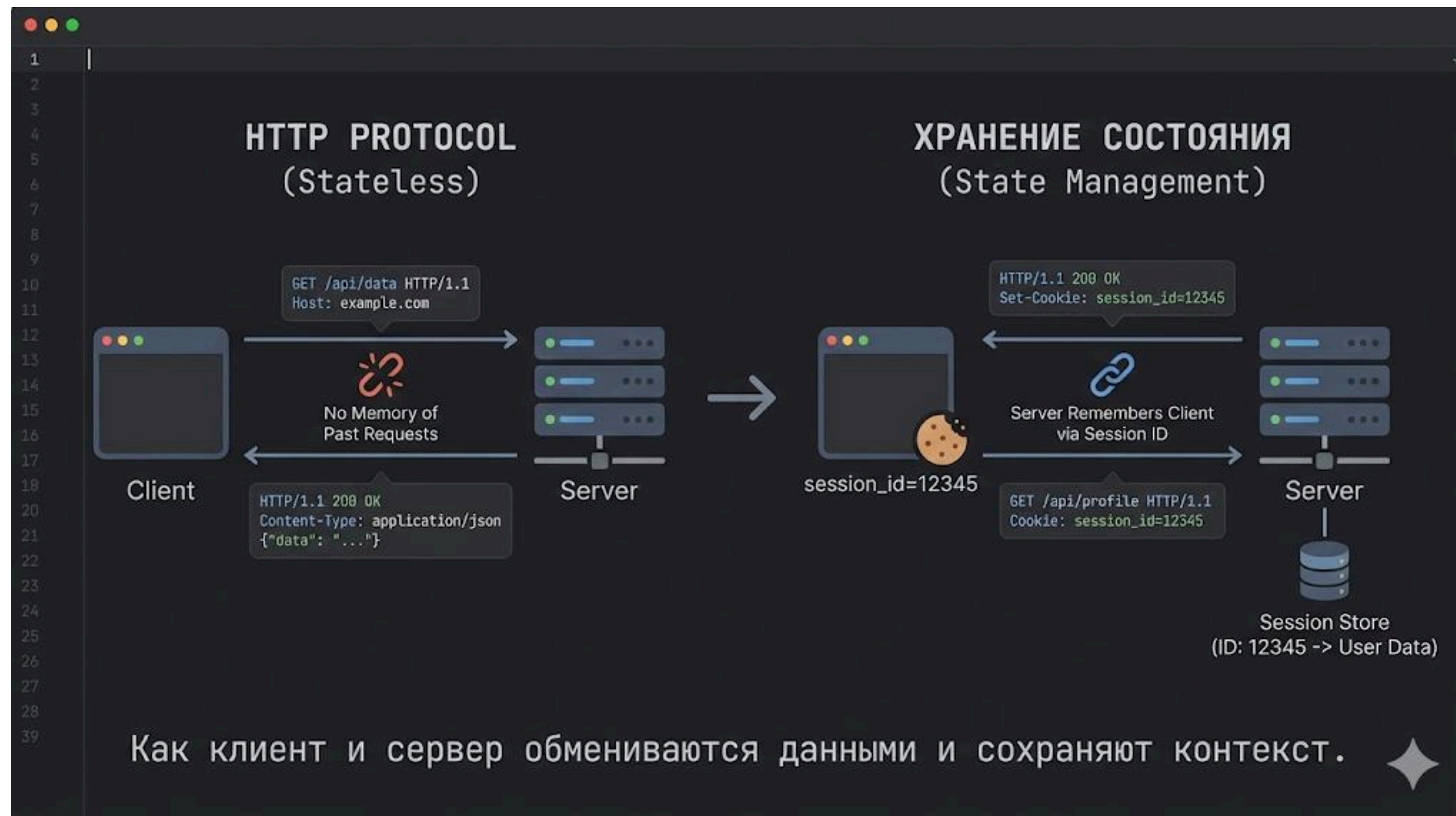
КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ

2026 Г.



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?

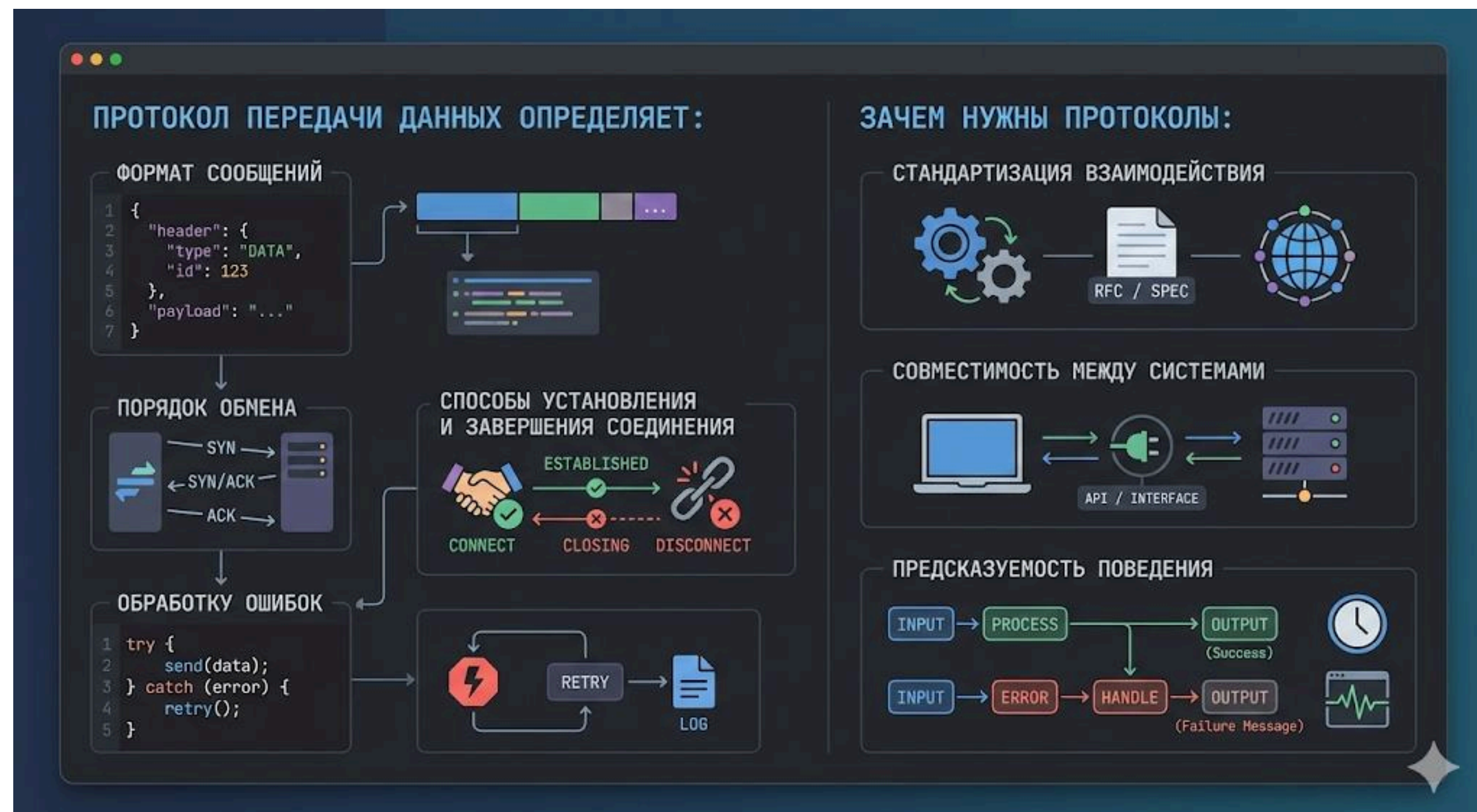


HTTP ПРОТОКОЛ И ХРАНЕНИЕ СОСТОЯНИЯ

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ

ЧТО ТАКОЕ ПРОТОКОЛ ПЕРЕДАЧИ ДАННЫХ



1. Протокол передачи данных — это набор правил, определяющих:

- формат сообщений
- порядок обмена
- способы установления и завершения соединения

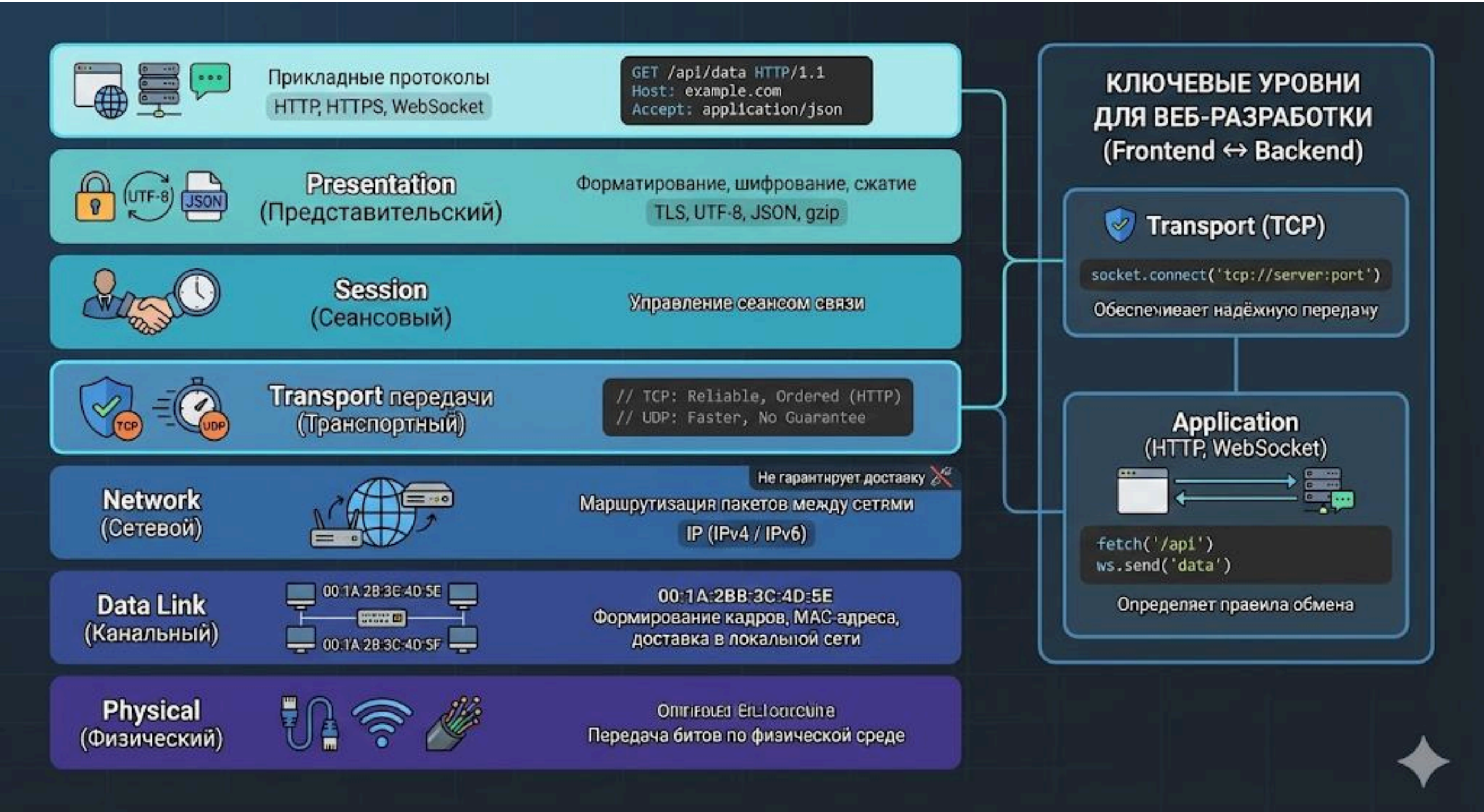
- обработку ошибок

2. Зачем нужны протоколы:

- стандартизация взаимодействия
- совместимость между системами
- предсказуемость поведения

УРОВНИ СЕТЕВОГО ВЗАИМОДЕЙСТВИЯ (МОДЕЛЬ OSI)

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ



1. Для понимания веб-взаимодействия важно знать уровни:

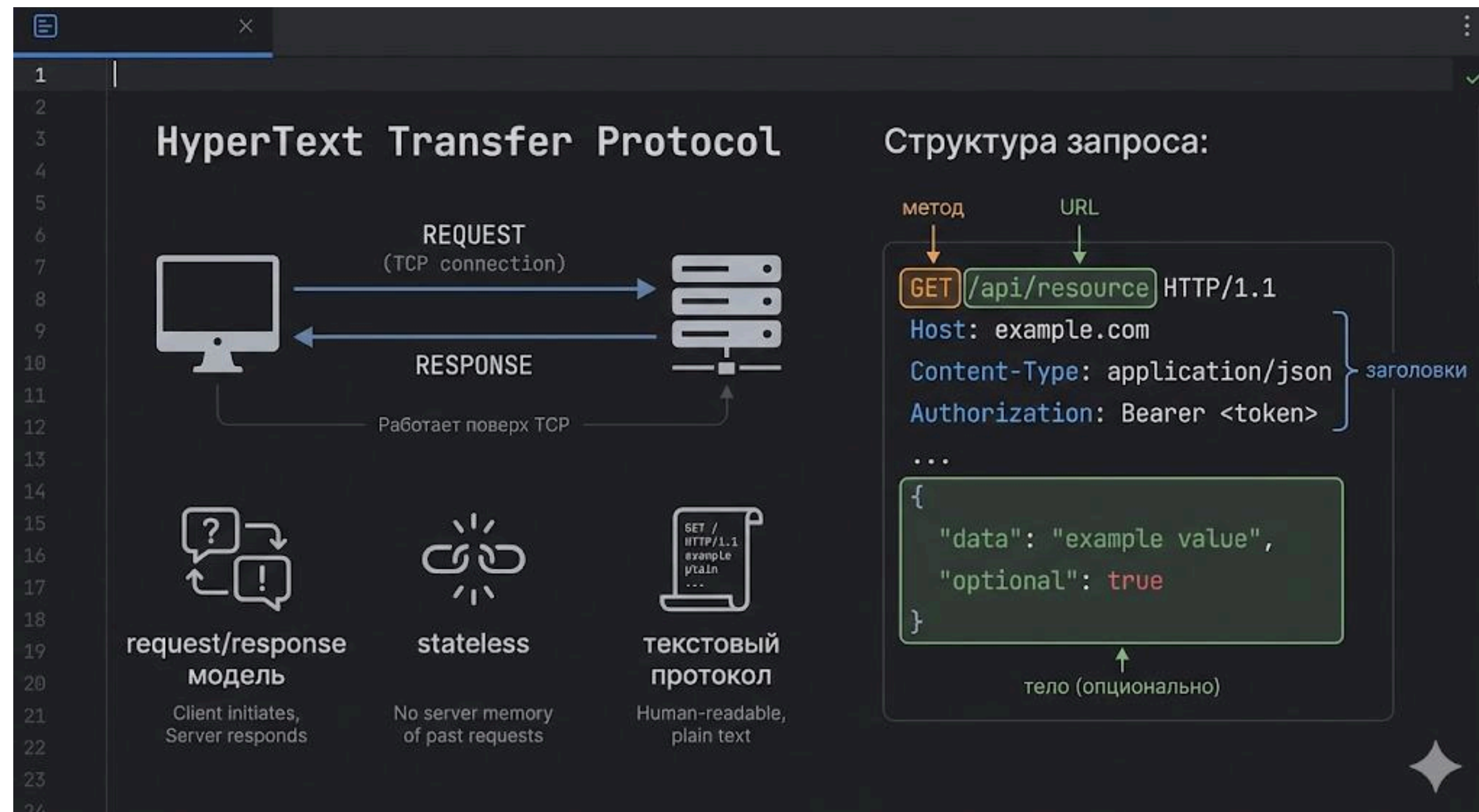
- a. Physical** - Передача битов по физической среде (кабель, Wi-Fi, оптоволокно).
- b. Data Link** - Формирование кадров, работа с MAC-адресами, доставка внутри локальной сети.
- c. Network** - Маршрутизация пакетов между сетями. Протокол: IP (IPv4 / IPv6). Не гарантирует доставку.
- d. Transport** - Надёжность передачи.
 - TCP – гарантирует доставку и порядок (используется HTTP).
 - UDP – быстрее, без гарантий.
- e. Session** - Управление сеансом связи (установка, поддержка, завершение).
- f. Presentation** - Форматирование данных, шифрование, сжатие. Примеры: TLS, UTF-8, JSON, gzip.
- g. Application** - Прикладные протоколы. Примеры: HTTP, HTTPS, WebSocket.

2. В веб-разработке ключевые уровни:

- **Transport (TCP)** – обеспечивает надёжную передачу
- **Application (HTTP, WebSocket)** – определяет правила обмена между frontend и backend

HTTP – ОСНОВНОЙ ПРОТОКОЛ ВЕБА

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ



1. HyperText Transfer Protocol

2. Работает поверх TCP.

3. Характеристики:

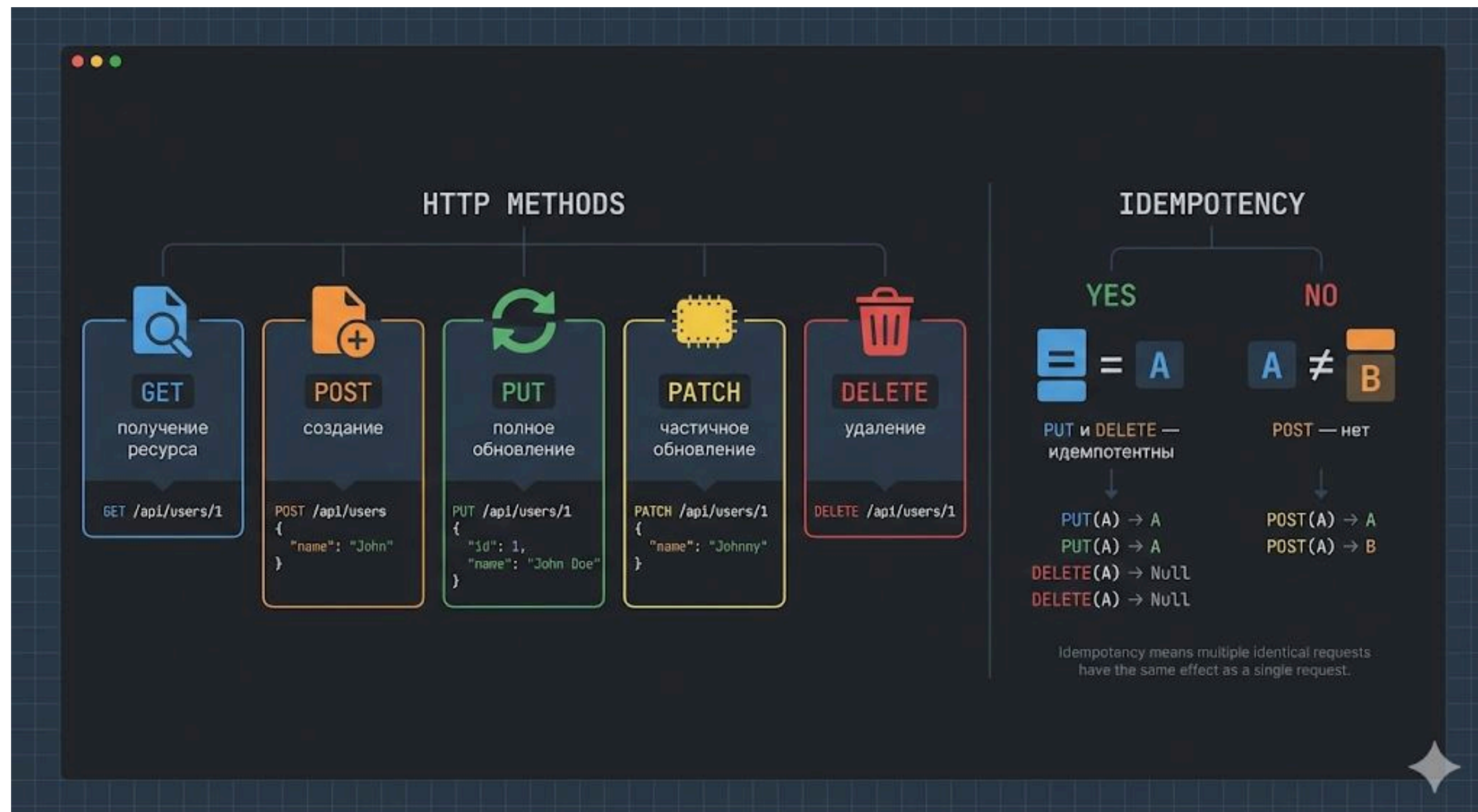
- request/response модель
- stateless - каждый запрос является уникальным и не зависит от данных, которые могут храниться в системе.
- текстовый протокол

4. Структура запроса:

- метод
- URL
- заголовки
- тело (опционально)

HTTP МЕТОДЫ

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ



1. Основные методы протокола HTTP:

- GET — получение ресурса
- POST — создание
- PUT — полное обновление
- PATCH — частичное обновление
- DELETE — удаление

2. Понятие idempotency:

- PUT и DELETE — идемпотентны
- POST — нет

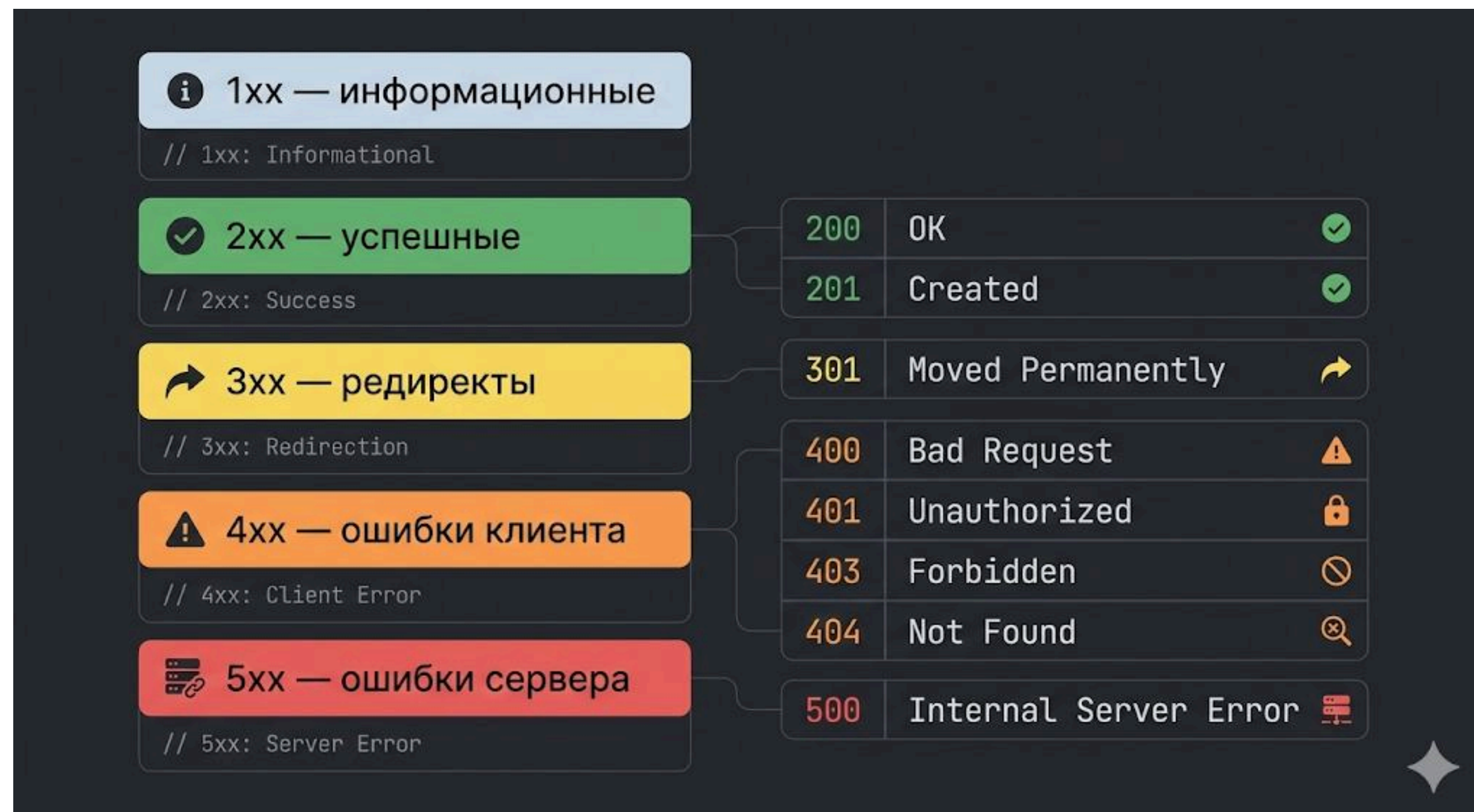
3. Идемпотентность означает, что многократное выполнение одного и того же запроса даёт тот же результат, что и однократное.

4. Метод PUT идемпотентен: повторное применение одного и того же запроса не меняет результат.

5. Метод DELETE также идемпотентен: повторное удаление уже удалённого ресурса не меняет состояние системы.

СТАТУС-КОДЫ HTTP

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ



1. Категории:

2. 1xx — информационные

2xx — успешные

3xx — редиректы

4xx — ошибки клиента

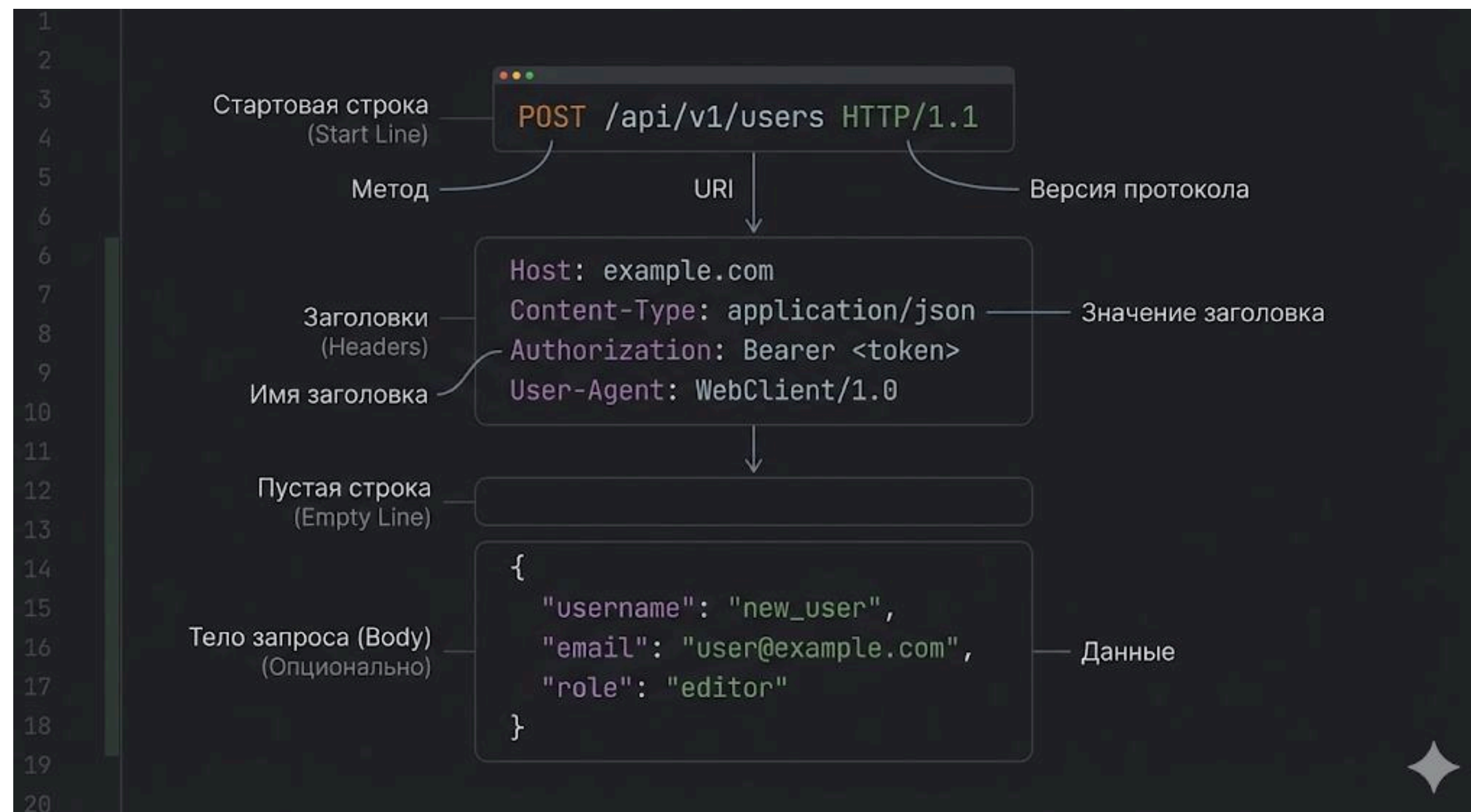
5xx — ошибки сервера

3. Примеры:

- 200 OK
- 201 Created
- 301 Moved Permanently
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

СТРУКТУРА HTTP-ЗАПРОСА

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ



1. HTTP-запрос – это сообщение клиента серверу, состоящее из 4 частей:

а. Стартовая строка (Request Line)

- i. Метод (GET, POST, PUT, DELETE и др.)
- ii. Путь к ресурсу
- iii. Версию протокола

б. Заголовки (Headers)

- i. Host: example.com - информацию о клиенте
- ii. Content-Type: application/json - формат данных
- iii. Authorization: Bearer token - авторизацию
- iv. Accept: application/json - тип ожидаемого ответа

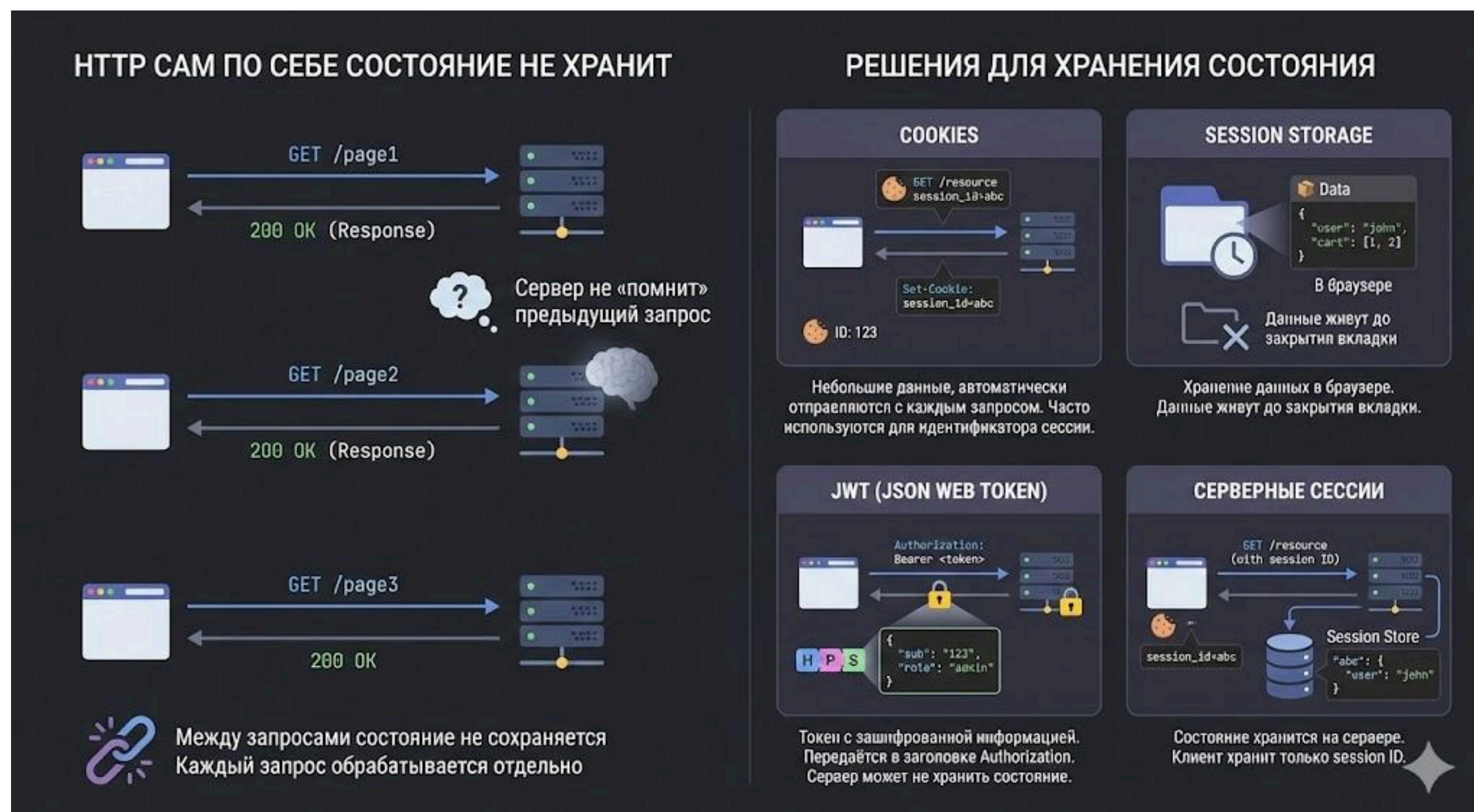
с. Пустая строка - Отделяет заголовки от тела запроса.

д. Тело запроса (Body) – опционально

- i. Передаётся в POST / PUT / PATCH:
- ii. Формат определяется заголовком Content-Type.

HTTP И ХРАНЕНИЕ СОСТОЯНИЯ

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ



1. HTTP сам по себе состояние не хранит — его реализуют через дополнительные механизмы поверх протокола.

а. Каждый запрос обрабатывается отдельно

б. Сервер не «помнит» предыдущий запрос

в. Между запросами состояние не сохраняется

2. Решения для хранения состояния

а. Cookies

- Небольшие данные, автоматически отправляются с каждым запросом

- Часто используются для идентификатора сессии

б. Session Storage

- Хранение данных в браузере

- Данные живут до закрытия вкладки

в. JWT (JSON Web Token)

- Токен с зашифрованной информацией

- Передаётся в заголовке Authorization

- Сервер может не хранить состояние

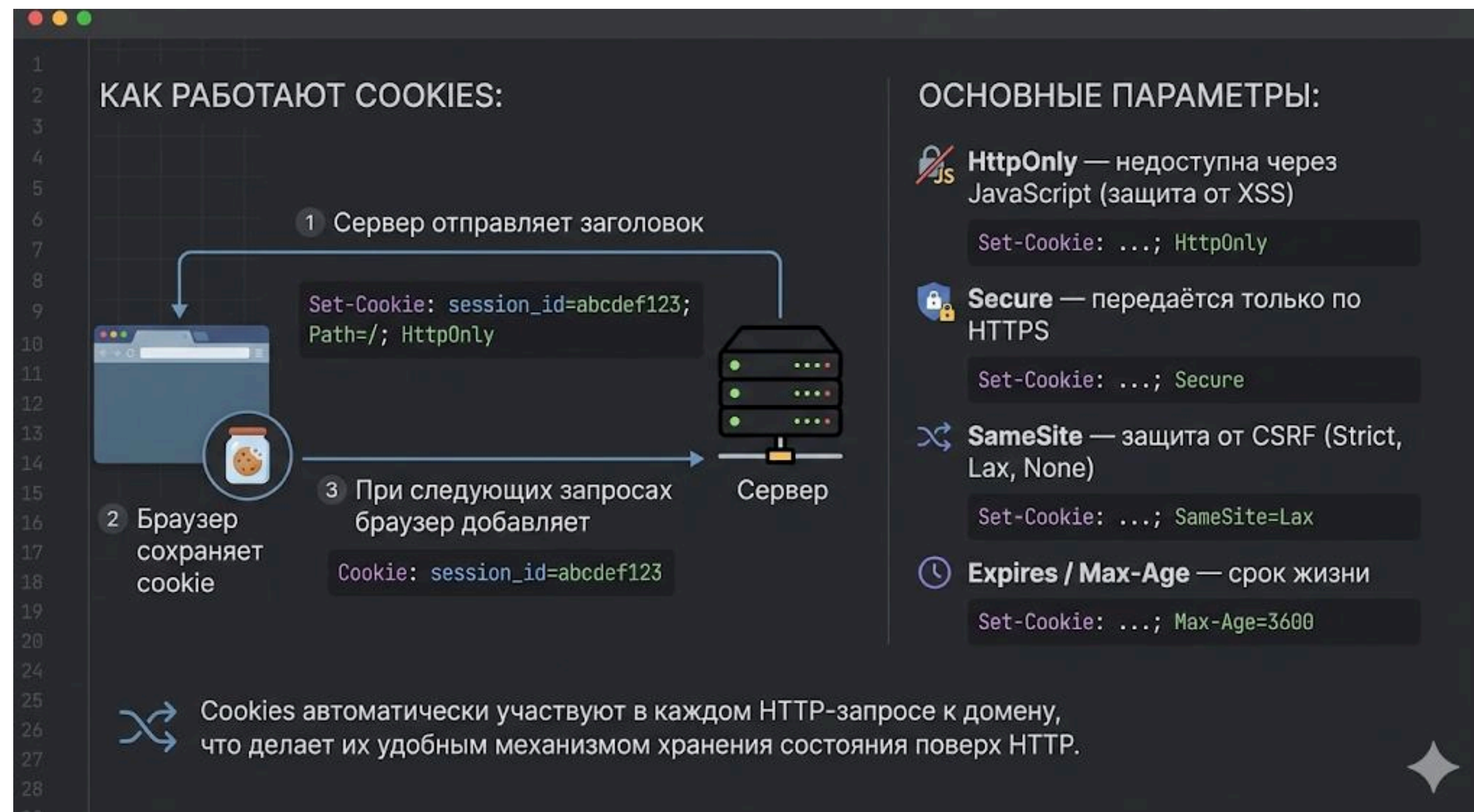
г. Серверные сессии

- Состояние хранится на сервере

- Клиент хранит только session ID

COOKIES

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ



1. Cookies — это небольшие данные, которые сервер отправляет браузеру, а браузер автоматически прикрепляет к каждому последующему запросу к тому же домену.

2. Как работают

а. Сервер отправляет заголовок:

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Lax
```

б. Браузер сохраняет cookie.

с. При следующих запросах браузер добавляет:

```
Cookie: sessionId=abc123
```

3. Основные параметры

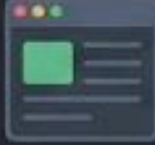
- HttpOnly — недоступна через JavaScript (защита от XSS)
- Secure — передаётся только по HTTPS
- SameSite — защита от CSRF
- Expires / Max-Age — срок жизни

4. Cookies автоматически участвуют в каждом HTTP-запросе к домену, что делает их удобным механизмом хранения состояния поверх HTTP

SESSION STORAGE

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ

ОСОБЕННОСТИ

-  Данные хранятся на стороне клиента
-  Привязаны к конкретной вкладке
-  Удаляются при её закрытии
-  Не отправляются автоматически на сервер

API БРАУЗЕРА

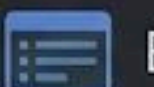
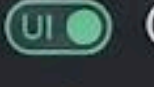
```
// Сохранить данные
sessionStorage.setItem('key', 'value');

// Получить данные
const data = sessionStorage.getItem('key');

// Удалить данные
sessionStorage.removeItem('key');

// Очистить всё
sessionStorage.clear();
```

КОГДА ИСПОЛЬЗУЕТСЯ

-  Временные данные формы
-  Состояние интерфейса
-  Одноразовые токены
-  Данные текущей сессии пользователя

1. Session Storage – это механизм хранения данных в браузере, который сохраняет состояние только в рамках одной вкладки.

2. Особенности

- Данные хранятся на стороне клиента
- Привязаны к конкретной вкладке
- Удаляются при её закрытии
- Не отправляются автоматически на сервер

3. API браузера

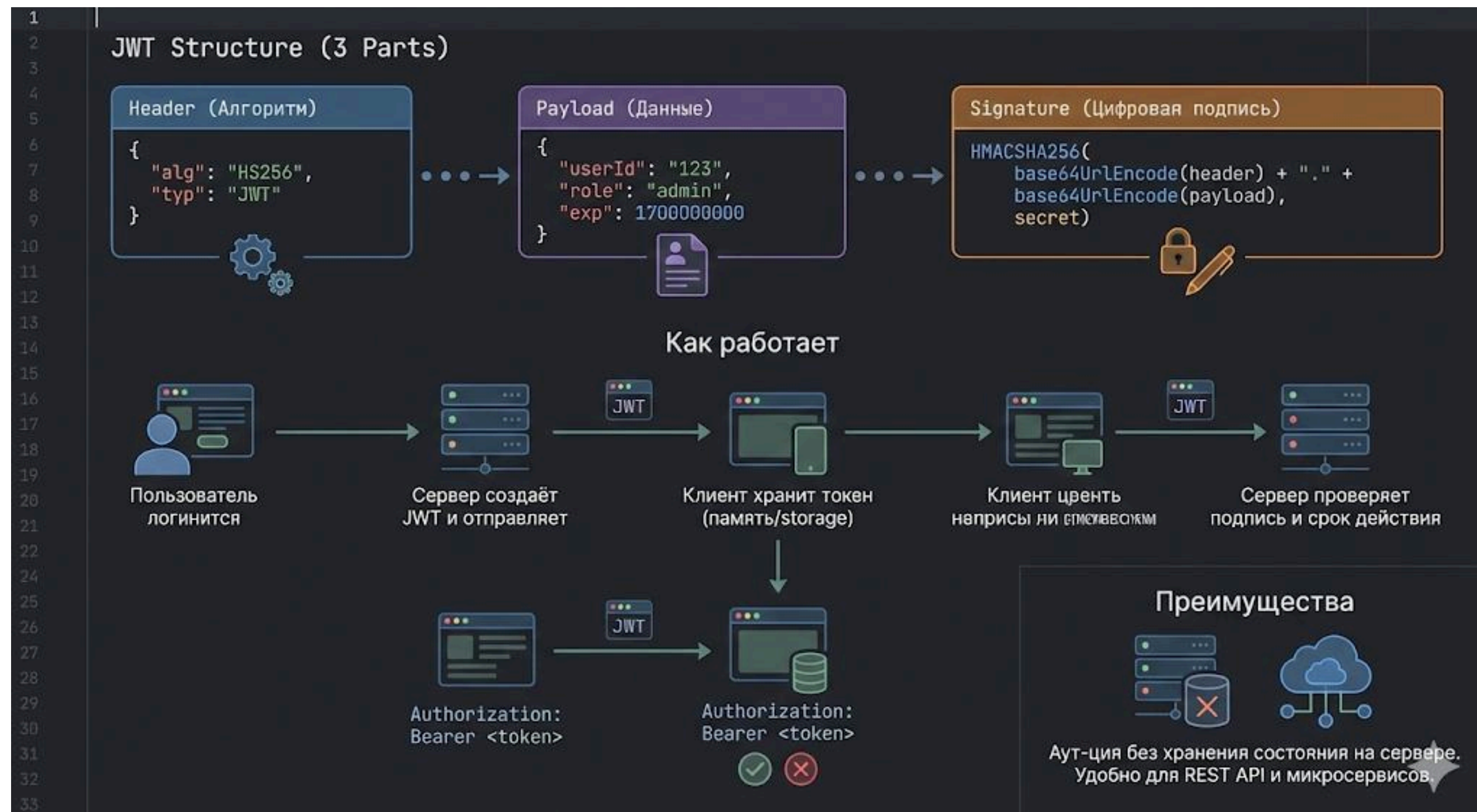
```
sessionStorage.setItem("token", "12345");
const token = sessionStorage.getItem("token");
sessionStorage.removeItem("token");
```

4. Когда используется

- Временные данные формы
- Состояние интерфейса
- Одноразовые токены
- Данные текущей сессии пользователя

JWT (JSON WEB TOKEN)

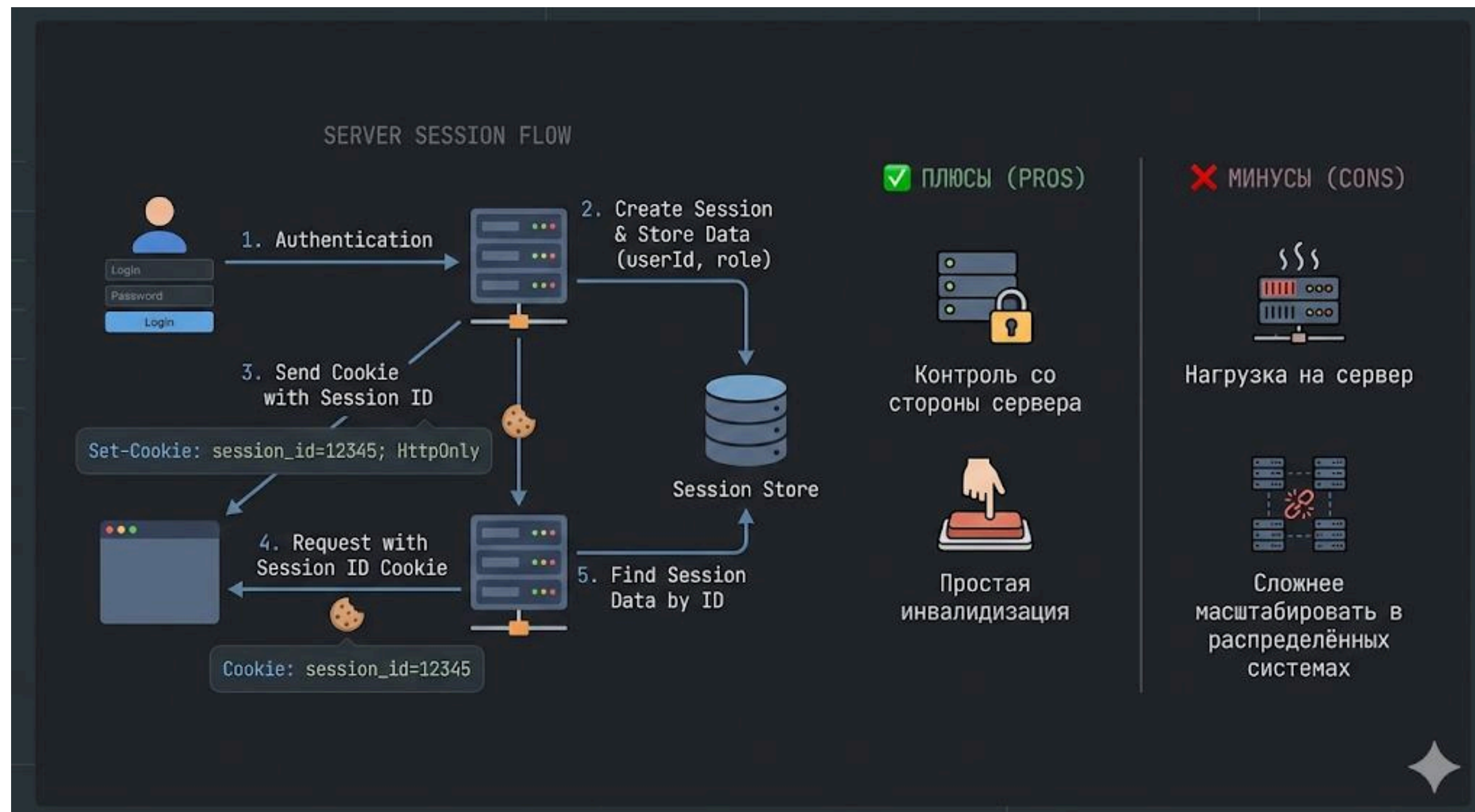
КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ



1. JWT — это компактный токен для передачи информации между клиентом и сервером в подписанном виде.
2. JWT состоит из 3 частей:
 - a. Header — алгоритм подписи (например, HS256)
 - b. Payload — данные (userId, role, exp и др.)
 - c. Signature — цифровая подпись
3. Как работает
 - a. Пользователь логинится
 - b. Сервер создаёт JWT и отправляет клиенту
 - c. Клиент хранит токен (например, в памяти или storage)
 - d. В каждом запросе отправляет:
 - i. Authorization: Bearer <token>
 - e. Сервер проверяет подпись и срок действия
 - f. JWT позволяет реализовать аут-цию без хранения состояния на сервере, что удобно для REST API и микросервисной архитектуры.

СЕРВЕРНЫЕ СЕССИИ

КАК КЛИЕНТ И СЕРВЕР ОБМЕНИВАЮТСЯ ДАННЫМИ



1. Серверная сессия — классический способ хранения состояния пользователя, при котором данные хранятся на сервере, а клиент содержит только идентификатор сессии.

- а. Пользователь проходит аутентификацию
- б. Сервер создаёт сессию и сохраняет данные (userId, роль и др.)
- в. Сервер отправляет клиенту cookie с session ID
- г. При каждом запросе клиент автоматически отправляет cookie
- д. Сервер по session ID находит данные сессии

2. Плюсы:

- Контроль со стороны сервера
- Простая инвалидизация

3. Минусы:

- Нагрузка на сервер
- Сложнее масштабировать в распределённых системах



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?